



SENSEAI

SenseAI

A Multi-Agent Investment Committee

Four data-driven AI analysts debate a ticker across multiple rounds, change their minds on evidence — not on the headcount — and a Chair synthesizes the call. Every decision clears an LLM compliance review and a layer of deterministic guardrails before a paper trade fires, fully traced in W&B Weave.

TEAM

SenseAI

TEAM MEMBERS

Radhika Khurana

Jeet S Swadia

REPOSITORY

github.com/khuranaradhika/senseai

Contents

PART I — SYSTEM OVERVIEW

- 1** Executive Summary
- 2** How It Works (High Level)
- 3** Setup & Running the System
- 4** LLM Configuration & Tunable Knobs

PART II — ARCHITECTURE (SHIPPED SYSTEM)

- 1** The Pipeline
- 2** Module Map
- 3** Data Model
- 4** LLM Layer
- 5** Safety: The Guardrail Layer
- 6** Live Event Contract (SSE)
- 7** Frontend
- 8** Observability
- 9** Tests

PART III — ROADMAP: V2 RE-ARCHITECTURE

- 1** Target Architecture & Locked Decisions
- 2** Phased Build Plan
- 3** Phase 0 — Foundation & Guardrails (Checklist)
- 4** Risks & Mitigations

PART IV — ENGINEERING LEARNINGS & LIMITATIONS

1 Engineering Learnings

2 Limitations & Sponsor Tools

1 • Executive Summary

SenseAI is a multi-agent **investment committee**. Four data-driven AI analysts — Bull, Bear, Risk, and Macro — debate a single ticker across multiple rounds. They are built to change their minds on evidence rather than on the headcount, and a Chair agent moderates every round and synthesizes the final call.

The decision then passes an LLM compliance review followed by a layer of **deterministic guardrails** enforced in code before a live Alpaca **paper** trade fires. Every step is traced in W&B Weave, logged as W&B metrics, and journaled for later outcome scoring against forward prices.

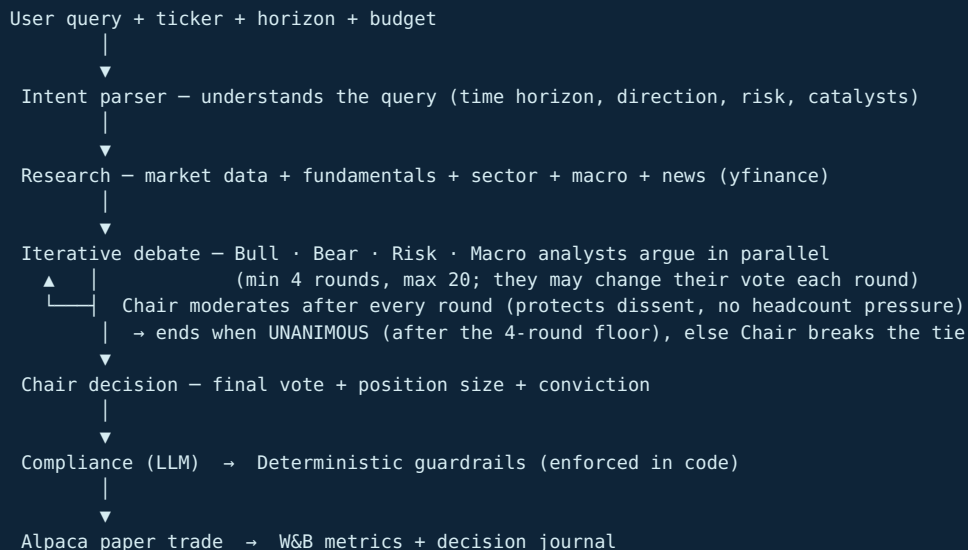
Scope. Research / demo project on **paper trading** only. It is not investment advice and has no proven track record — see the Limitations section.

What makes it distinctive

- **Debate, not a vote count.** Analysts are analytical lenses prompted to vote honestly from evidence; the system is explicitly designed to resist herding.
- **Code enforces, prompts advise.** Every hard limit lives in a pure, unit-tested `evaluate()` function that the LLMs cannot override.
- **Full observability.** Each agent and tool call is a Weave op, so a run is a complete nested trace from intent to trade.
- **Closes the loop.** A decision journal records entry prices and later scores them against forward prices for hit-rate and calibration.

2 · How It Works (High Level)

A single run flows through intent parsing, one research pass, an iterative debate, a Chair decision, compliance review, and a deterministic execution gate:



Debate dynamics

- **Data-driven, not advocacy.** Bull / Bear / Risk / Macro are analytical lenses, each prompted to vote honestly from evidence — not to defend a predetermined side.
- **Minds change on evidence, not headcount.** Prompts explicitly forbid conforming to the majority; the Chair never cites the tally and pushes the majority to rebut the minority's best point.
- **Consensus = unanimous**, but only allowed to end the debate after `MIN_DEBATE_ROUNDS`. Genuine splits ride to `MAX_DEBATE_ROUNDS` and the Chair breaks the tie.
- **Horizon shapes weighting.** Intraday leans on technicals / momentum; long horizons lean on fundamentals / secular trends.

3 · Setup & Running the System

Setup

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
cp .env.example .env      # then fill in your keys
```

Environment (`.env`)

```
# Weights & Biases (Inference + Weave) — https://wandb.ai/authorize
WANDB_API_KEY=your_wandb_key

# Alpaca paper trading — https://app.alpaca.markets (Alpaca's standard APCA_* names)
APCA_API_KEY_ID=your_alpaca_key_id
APCA_API_SECRET_KEY=your_alpaca_secret
APCA_BASE_URL=https://paper-api.alpaca.markets
```

The Alpaca client also accepts the legacy `ALPACA_API_KEY` / `ALPACA_SECRET_KEY` / `ALPACA_BASE_URL` names as a fallback. `.env` is gitignored — never commit real keys.

Run it

Web UI (recommended):

```
python server.py      # → http://127.0.0.1:8000
```

Enter a ticker, an investment query, a horizon (1 day → 20 years), and a max budget, then **CONVENE**. The debate streams live: a vote-trajectory matrix, collapsible rounds, Chair moderation, the verdict, guardrail checks, and a W&B run link.

CLI (demo tickers):

```
python main.py
```

Tests / smoke runner:

```
python test.py          # fast checks (no LLM): tools, guardrails, journal
python test.py committee NVDA SHORT --quick # one full debate (2 rounds)
python -m pytest tests/ -q      # unit tests (parsing + guardrails)
```

Score past decisions against forward prices (run later, once time has passed):

```
python -m src.core.journal review
```

4 · LLM Configuration & Tunable Knobs

LLM configuration

All reasoning runs through the **W&B Inference API** (OpenAI-compatible), <https://api.inference.wandb.ai/v1>. Two tiers are defined in `src/core/config.py`:

Tier	Model	Used by
Fast	deepseek-ai/DeepSeek-V4-Flash	the four analysts
Smart	deepseek-ai/DeepSeek-V4-Pro	intent parser, Chair, compliance

Override with the `FAST_MODEL` / `SMART_MODEL` env vars. List available models:

```
curl https://api.inference.wandb.ai/v1/models -H "Authorization: Bearer $WANDB_API_KEY"
```

Configuration knobs

All thresholds live in `src/core/config.py` and are env-overridable:

Env var	Default	Meaning
MIN_DEBATE_ROUNDS	4	floor on debate rounds before consensus can end it
MAX_DEBATE_ROUNDS	20	hard ceiling on rounds
CONVICTION_MIN	0.70	execution gate — Chair conviction
AVG_CONF_MIN	0.60	execution gate — average analyst confidence
MAX_POSITION_USD	1000	global hard position cap (UI "Max Buy" overrides per-run)
MAX_PORTFOLIO_PCT	0.10	max fraction of equity per position
MAX_DAILY_TRADES	5	daily trade cap
LLM_TIMEOUT_S / LLM_MAX_RETRIES	60 / 3	LLM resilience

PART II — ARCHITECTURE (SHIPPED SYSTEM)

This part describes how SenseAI is built end to end: the pipeline, every module's role, the data model, the live event contract, the safety layer, and observability.

5 • The Pipeline

A single run is orchestrated by `run_committee()` in `src/core/orchestrator.py`:

```
run_committee(query, ticker, config, emit, intent)
|
|— 0. Understand the query
|   intent = parse_intent(query, ticker)           # src/agents/intent.py
|   directive = intent_directive(intent)          # horizon → how to weigh data
|   time_ctx = time_context()                     # today's date + market open/closed
|
|— 1. Research (fetched once, reused all rounds)
|   market_data = fetch_market_data(ticker)       # price, RSI(Wilder), MACD, range, volume
|   fundamentals = fetch_fundamentals(ticker)     # P/E, fwd P/E, PEG, growth, margins, ROE...
|   sector = fetch_sector(ticker)                 # sector + relative strength vs sector ETF
|   macro = fetch_macro()                         # S&P, Nasdaq, VIX, 10Y, DXY
|   news = fetch_news(ticker)                    # recent headlines
|   research = fundamentals + sector + macro + news (one brief)
|
|— 2-3. Iterative debate (for round in 1..MAX_DEBATE_ROUNDS)
|   parallel: analyst_round(name, ...) for name in [Bull, Bear, Risk, Macro]
|       each sees: market data, research brief, directive (horizon), time_ctx,
|                 the other analysts' latest positions, the Chair's last note,
|                 and its own previous position → returns vote + thesis + confidence
|   chair_note = chair_moderate(...)              # active moderation after EVERY round
|   unanimous = (all four votes identical)
|   stop if round >= MIN_DEBATE_ROUNDS and unanimous → consensus
|   stop if round == MAX_DEBATE_ROUNDS → no consensus
|
|— 4. Chair decision
|   final_vote, rationale, size, conviction = chair_decide(state, config, consensus_vote, ...)
|   consensus → ratify it + size by conviction; no consensus → break the tie
|
|— 5. Compliance (LLM, advisory)
|   approved, reason, adjusted_size = compliance_agent(state, config)
|
|— 6. Execution (deterministic guardrails are the final gate)
|   guardrails.evaluate(...) → approve/clamp/block
|   if approved and side in {buy, sell}: alpaca.execute_trade(...)
|   record_decision(state) # journal
|   log_run_to_wandb(state) # metrics (server path)
```


6 · Module Map

```
main.py          CLI entry (runs demo tickers, logs to W&B, journals)
server.py        FastAPI server: serves the UI + /run SSE stream + /logo.png
test.py          smoke/integration runner (tools, intent, guardrails, journal, committee)
ui/index.html    single-page dashboard (SSE client, no build step)

src/core/
  orchestrator.py  run_committee() – the whole pipeline above
  config.py        CONFIG singleton: all thresholds + model tiers (env-overridable)
  schemas.py       dataclasses: Intent, MarketData, AgentFinding, DebateState, ...
  llm.py           W&B Inference client: _call (retry/backoff), call_llm, call_typed
  parsing.py       parse_structured(): JSON extract → Pydantic validate → reprompt-once
  guardrails.py    evaluate(): deterministic pre-trade checks; gather_inputs()
  clock.py         time_context(): today + market-open grounding
  journal.py       SQLite decision log + review() forward-return scoring
  evaluation.py    score_debate(), log_debate_summary(), log_run_to_wandb()

src/agents/
  intent.py        parse_intent() + horizon buckets/guidance + intent_directive()
  analyst.py       PERSONAS (Bull/Bear/Risk/Macro) + analyst_round()
  chair_agent.py   chair_moderate() (per-round) + chair_decide() (final)
  compliance_agent.py LLM compliance review (advisory)

src/tools/
  market_data.py  yfinance price + Wilder RSI + MACD + range/volume + as-of date
  fundamentals.py yfinance valuation/growth/profitability/balance-sheet
  sector.py        sector classification + 1-mo relative strength vs sector ETF
  macro.py         indices, VIX, 10Y yield, DXY (level + 1-mo change)
  news.py          recent headlines
  alpaca.py        account / positions / clock / orders-today / execute_trade
```

7 • Data Model

src/core/schemas.py

- **Intent** — `horizon_bucket` (INTRADAY/SHORT/MEDIUM/LONG/VERY_LONG), `horizon_detail`, `direction`, `risk_tolerance`, `catalysts`, `constraints`, `interpretation`.
- **MarketData** — price, change %, volume/avg, 52-wk range, P/E, market cap, RSI, MACD, `summary`, `as_of` (latest bar date).
- **AgentFinding** — `agent_name`, `thesis`, `key_points`, `confidence`, `vote` (BUY/SELL/HOLD), `round`, `changed_vote`, optional `rebuttal`.
- **DebateState** — the run's full state: `intent`, `market_data`, `transcript` (list of rounds, each a list of findings), `chair_notes`, `rounds_run`, `consensus_reached`, `final_vote`, `conviction`, `position_size`, compliance fields, `trade_executed`, `trade_result`. (`findings` = round 1, `rebuttals` = last round — kept for backward-compatible logging.)

Agent outputs are validated into **Pydantic** models (`AnalystOutput`, `ChairModeration`, `ChairDecision`, `ComplianceOutput`, `IntentOutput`) via `call_typed`, then mapped to the dataclasses above.

8 · LLM Layer

src/core/llm.py

- `_call` posts to the W&B Inference `/chat/completions` endpoint with retry + exponential backoff (`LLM_MAX_RETRIES` , `LLM_TIMEOUT_S`); 4xx surfaces immediately, 5xx/timeouts retry.
- `call_typed(system, user, Model, smart=...)` is the safe path every agent uses: `call` → `parse_structured` (strip fences → `json.loads` → `Model.model_validate`) → on failure, **reprompt once** with the error → else raise typed `StructuredParseError` . A single malformed response never crashes a run.
- Two model tiers (`smart` flag) selected from `CONFIG` .

9 · Safety: The Guardrail Layer

src/core/guardrails.py

`evaluate(...)` is a **pure, network-free** function (the orchestrator passes in account / positions / clock / daily count via `gather_inputs()`), so it is fully unit-tested. It runs **after** the LLM compliance review and is the final word:

Check	Blocks when
<code>actionable_direction</code>	vote is HOLD (no order by design)
<code>confidence_gate</code>	conviction < <code>CONVICTION_MIN</code> or avg confidence < <code>AVG_CONF_MIN</code>
<code>market_open</code>	market is closed
<code>daily_trade_cap</code>	today's order count ≥ <code>MAX_DAILY_TRADES</code>
<code>no_conflicting_position</code>	an open position already exists in the symbol
<code>sufficient_sizing</code>	clamped notional ≤ 0

Final size = `min(proposed, MAX_POSITION_USD, MAX_PORTFOLIO_PCT × equity, buying_power)` . Broker unreachable → **block for safety**. Per-run budget (UI "Max Buy") overrides the global cap via `dataclasses.replace(CONFIG, max_position_usd=...)` .

10 · Live Event Contract (SSE)

The UI opens an `EventSource` to `GET /run?ticker=&query=&horizon=&max_position=`. The server runs the committee in a worker thread; `run_committee`'s `emit` callback pushes typed JSON events onto a queue that the SSE generator streams.

Event	Payload (key fields)
<code>intent</code>	<code>horizon_bucket</code> , <code>direction</code> , <code>risk_tolerance</code> , <code>catalysts</code> , <code>constraints</code> , <code>interpretation</code>
<code>status</code>	<code>message</code>
<code>market_data</code>	<code>price</code> , <code>change_pct</code> , <code>rsi</code> , <code>macd</code>
<code>research</code>	<code>fundamentals</code> , <code>sector</code> , <code>macro</code> , <code>news[]</code>
<code>round_start</code>	<code>round</code>
<code>finding</code>	<code>round</code> , <code>agent</code> , <code>vote</code> , <code>confidence</code> , <code>thesis</code> , <code>key_points</code> , <code>changed_vote</code>
<code>chair_note</code>	<code>round</code> , <code>note</code> , <code>vote_counts</code> , <code>unanimous</code>
<code>votes</code>	<code>final_vote</code> , <code>position_size</code> , <code>chair_decision</code> , <code>consensus</code> , <code>rounds</code>
<code>compliance</code>	<code>approved</code> , <code>reason</code> , <code>position_size</code>
<code>execution</code>	<code>executed</code> , <code>final_vote</code> , <code>position_size</code> , <code>result</code> , <code>blocked_by</code> , <code>guardrails[]</code>
<code>metrics</code>	<code>scores</code> , <code>wandb_url</code>
<code>complete</code>	—

The same `emit` interface means the CLI runs the identical pipeline with no callback.

11 · Frontend

ui/index.html

Single file, no build step. Brand: **SenseAI** (Chakra Petch wordmark, deep-navy + cyan to match the logo; BUY-green / SELL-red / HOLD-amber for votes). Live views:

- **Sticky status bar** — pipeline stage + live round + running tally + consensus pill.
- **Vote-trajectory matrix** — agents × rounds grid; each cell is the vote (colored) with a  when it changed; shows the "minds changing" story at a glance.
- **Collapsible round groups** — each round is a `<details>` block; older rounds auto-collapse so a 20-round run stays scannable.
- **Research card** — fundamentals / sector / macro / news.
- **Verdict + trade card** — final call, size, consensus-vs-tiebreak, and every guardrail check.

12 · Observability

- **Weave** (`weave.init("investment-committee")`): `@weave.op` on the orchestrator, every agent, `call_llm / call_typed`, and every tool → a full nested trace per run (intent → research → each round's analysts + Chair → decision → compliance → guardrails → trade).
- **W&B metrics** (`evaluation.log_run_to_wandb`): one run per decision with decision + process-quality metrics (`score_debate` : rounds, consensus, vote_changes, initial/final vote diversity, avg final confidence). Fires from **both** the UI and CLI.
- **Decision journal** (`journal.py`): SQLite store (`data/decisions.db`) of every decision + entry price; `review()` fetches the forward price, computes the realized move, labels correctness, and aggregates hit-rate / returns-by-vote / conviction calibration → logged to W&B as a `review` run.

The W&B / Weave **project** is named `investment-committee` (internal identifier).

13 · Tests

- `tests/test_parsing.py` — fence-strip, valid parse, reprompt-repair, hard-fail, schema violation.
- `tests/test_guardrails.py` — every guardrail branch (gate, market closed, daily cap, conflict, clamping to each cap).
- `test.py` — runnable smoke checks across tools / intent / guardrails / journal / a full committee.

Parts I and II document the **shipped** committee system. This part captures the planned v2 re-architecture: a move from the static bull/bear/chair/compliance committee to a **dynamic, routed, self-critiquing pipeline**, optimized for a judged demo where the **Weave story is the product** — every routing decision and critic rejection is a visible trace, so judges see the reasoning behind the harness in real time.

14 · Target Architecture & Locked Decisions

Decisions locked

- **Migration:** Full replacement. Retire `orchestrator.py` + bull/bear/risk/macro/chair/compliance agents. Reuse infra only: `src/core/llm.py` (W&B DeepSeek), `src/tools/alpaca.py`, `server.py` SSE, Weave.
- **Tools:** yfinance + free sources (no paid keys).
- **Goal:** Hackathon / judged demo — MVP scope, reliability over a handful of live runs, rich traces.
- **Loop limits (conservative):** Critic ≤ 2 revision rounds per finding. Execute only if `conviction  $\geq$  0.70 AND avg_specialist_confidence  $\geq$  0.60`.

Target architecture

```

UserQuery
↓
PlannerAgent — reads query, selects which specialists to invoke [TRACE: routing decision + rationale]
  ↓ (dynamic routing — 1..4 specialists)
[MacroAgent] [SectorAgent] [CompanyAgent] [SentimentAgent] (parallel, each does tool use)
↓
CriticAgent — scores each finding; weak ones go back for revision [TRACE: every accept/reject + reason]
  ↑———— feedback loop (max 2 rounds/finding) —————↓
  ↓
ArbiterAgent — synthesizes approved findings → structured thesis
↓
ConvictionScorerAgent — conviction score (0–1) + position sizing
  ↓ (deterministic gate: conviction  $\geq$  0.70 AND avg_conf  $\geq$  0.60)
ExecutionAgent — deterministic guardrails → Alpaca paper trade

```

Specialist → tool mapping (yfinance + free)

Specialist	Question it answers	Tools
MacroAgent	Is the macro regime a tailwind or headwind?	indices (<code>^GSPC</code>), VIX (<code>^VIX</code>), 10y yield (<code>^TNX</code>), DXY
SectorAgent	How does the name look vs its sector/peers?	sector lookup, peer list, relative performance
CompanyAgent	Are the fundamentals + technicals sound?	yfinance <code>.info</code> (P/E, margins, growth), price/RSI/MACD
SentimentAgent	What's the news/sentiment posture?	yfinance <code>.news</code> / free RSS headlines

Each tool is a `@weave.op`, so tool calls show up as traces. Specialists select tools via a **bounded function-calling loop (≤ 3 tool calls each)**; if the model returns no tool call, the system falls back to pre-fetched context so a run never stalls.

Data model (full rewrite, Pydantic)

- `Query{ ticker, question, raw }`
- `RoutingDecision{ selected, skipped, rationale }`
- `Evidence{ source, key, value, note }` — what a finding actually cited
- `SpecialistFinding{ agent, signal (bullish/bearish/neutral), thesis, key_points, evidence[], confidence, revision }`
- `Critique{ finding_agent, approved, score, issues[], revision_request? }`
- `Thesis{ direction (BUY/SELL/HOLD), summary, supporting[], opposing[], risks[] }`
- `ConvictionResult{ conviction, position_size_usd, sizing_rationale }`
- `GuardrailCheck{ name, passed, detail }`
- `ExecutionResult{ executed, side, notional_usd, order_id?, blocked_by?, checks[] }`
- `PipelineState{ query, routing, findings, critiques, thesis, conviction, execution, started_at }`

15 · Phased Build Plan

Phase 0 — Foundation & guardrails

Reusable substrate that every agent depends on (detailed checklist in Section 16).

- `config.py` — central thresholds + model tiers (kills stale `claude-sonnet` in `main.py`).
- `parsing.py` — `parse_structured(raw, Model)` : fence-strip → `json.loads` → Pydantic validate → one reprompt on failure → typed error.
- `llm.py` — add retry/backoff + timeout around the W&B call; keep `call_llm` / `call_llm_structured` signatures.
- `guardrails.py` — deterministic pre-trade checks. Thresholds enforced in code, never just in a prompt.
- `tests/` — pytest for parse-repair, threshold enforcement, position clamp, critic-loop termination.

Exit criteria: guardrails + parsing unit-tested green; thresholds enforced deterministically; malformed LLM JSON never crashes a run.

Phase 1 — Tools (`src/tools/`)

- Refactor `market_data.py` into a reusable price/technical tool.
- New `fundamentals.py`, `sector.py`, `news.py`, `macro.py` — each a `@weave.op` returning a small typed dict.
- A `ToolRegistry` mapping specialist → allowed tools + OpenAI-style tool schemas.

Exit criteria: each tool callable standalone, returns within timeout, degrades gracefully when yfinance is missing a field.

Phase 2 — Agents (`src/agents/`)

- `planner.py` — emit `RoutingDecision` (1–4 specialists + rationale). The headline dynamic-routing trace.
- `specialists/{macro,sector,company,sentiment}.py` — each runs its bounded tool-calling loop, returns a `SpecialistFinding` with cited evidence.
- `critic.py` — scores a finding; `approved=False` emits a `revision_request`. Calibrated to lower confidence / flag thin evidence.
- `arbiter.py` — synthesize approved findings → `Thesis`.
- `conviction.py` — `Thesis` + findings → `ConvictionResult` (vol-aware sizing bounded by config caps).
- `execution.py` — runs `guardrails.py`, then fires Alpaca paper trade only if all checks pass.

Exit criteria: each agent independently testable with a fixture state; outputs validate against schemas.

Phase 3 — Pipeline orchestration (`src/core/pipeline.py`)

Replaces `orchestrator.py`. `run_pipeline(query, ticker, config, emit=None)` : Planner → routing; parallel specialists (ThreadPoolExecutor) over the selected set only; critic loop per finding (≤ 2 rounds); arbiter → thesis; conviction → score + size; execution gate → guardrails → Alpaca. The `emit` callback fires a typed event at every step.

Exit criteria: full run from query → execution completes; Weave shows nested traces with routing + rejection reasoning.

Phase 4 — SSE events + UI

- `src/core/events.py` — typed event builders.
- `server.py /run` — new event types: `routing`, `tool_call`, `finding`, `critique`, `revision`, `thesis`, `conviction`, `execution`, `complete`.
- `ui/index.html` — stages become Planner → Route → Specialists → Critic → Arbiter → Conviction → Execute; new specialist cards; a **Critic rejections** panel; routing rationale banner.

Exit criteria: live demo streams routing + rejections visibly; reconnect/disconnect handled.

Phase 5 — Retire old system + polish

- Delete `orchestrator.py` + 6 legacy agents; update `main.py` CLI to `run_pipeline`.
- README + demo script (3 tickers: a clean BUY, a critic-forced revision, a HOLD/blocked-by-guardrail).
- Smoke test across the demo tickers.

Exit criteria: `python main.py` and `python server.py` both run the new pipeline end-to-end; no references to retired modules.

Out of scope (v2)

Live (non-paper) trading, multi-ticker portfolios, backtesting/P&L attribution, auth, paid data. Captured as future work.

16 · Phase 0 — Foundation & Guardrails (Checklist)

These are the guardrail todos to start on first — the substrate the whole v2 pipeline is built on. Every agent depends on safe parsing, central config, and deterministic guardrails. Build and test these before any pipeline agents.

1 · Central config — `src/core/config.py`

- Define one config object with all thresholds: `CONVICTION_MIN = 0.70` , `AVG_CONF_MIN = 0.60` , `MAX_CRITIC_ROUNDS = 2` , `MAX_POSITION_USD = 1000.0` , `MAX_PORTFOLIO_PCT = 0.10` , `MAX_DAILY_TRADES = 5` , `MAX_TOOL_CALLS = 3` ; model tiers FAST / SMART.
- Load overrides from env; sensible defaults baked in.
- Remove the stale `"model": "claude-sonnet-..."` config in `main.py` — a leftover from the W&B migration.

Done when: thresholds live in exactly one place and are imported, not duplicated.

2 · Safe structured-output layer — `src/core/parsing.py`

- `parse_structured(raw, model)` : fence-strip → `json.loads` → Pydantic `model_validate` .
- On failure: one reprompt including the validation error, then re-parse. On second failure: raise typed `StructuredParseError` .
- Replace every bare `json.loads(raw)` in the agents once Phase 2 lands.

Done when: a deliberately malformed LLM response is repaired or fails cleanly — never crashes a run.

3 · LLM resilience — `src/core/llm.py`

- Wrap the `requests.post` in retry w/ exponential backoff (3 attempts) on timeout / 5xx / connection error.
- Keep the existing 60s timeout and `call_llm` / `call_llm_structured` signatures stable.
- Optional: smart→fast model fallback on repeated failure.

Done when: a transient W&B inference blip retries instead of killing the pipeline.

4 · Deterministic guardrails — `src/core/guardrails.py`

`check_execution(thesis, conviction, account, config)` , enforced in code (not prompts): confidence gate (`conviction ≥ CONVICTION_MIN` AND `avg_specialist_confidence ≥ AVG_CONF_MIN`); position cap; portfolio cap (`≤ MAX_PORTFOLIO_PCT × equity`); buying power; market hours; duplicate/conflicting position; daily trade cap; HOLD → skip. Any failed check blocks the trade, with the failing check name surfaced to the UI/trace.

Done when: execution is impossible unless every deterministic check passes, regardless of what any LLM says.

5 · Critic-loop bound

- Enforce `MAX_CRITIC_ROUNDS = 2` with a deterministic counter in the pipeline (Phase 3).
- After the cap: accept-with-flag or drop the finding — never loop indefinitely.

6 · Tests — tests/ (pytest)

- `test_parsing.py` — fence-strip, valid parse, repair-on-bad-json, hard-fail path.
- `test_guardrails.py` — confidence gate blocks below threshold; position clamp; portfolio cap; HOLD skip; market-closed block.
- `test_critic_loop.py` — loop terminates at 2 rounds.

17 · Risks & Mitigations

Risk	Mitigation
LLM JSON drift	safe-parse + reprompt (Phase 0) makes it non-fatal
Critic infinite loop / cost	hard 2-round cap + tool-call cap
yfinance flakiness / rate limits	per-tool try/except + graceful degradation; cache within a run
W&B inference latency	retry/backoff + per-call timeout; fast tier for specialists, smart tier for critic/arbiter/conviction
Demo reliability	deterministic guardrails + fallbacks mean a run always reaches a verdict

PART IV — ENGINEERING LEARNINGS & LIMITATIONS

Engineering learnings from building the committee — the bugs that mattered, the design choices and why, and the failure modes that only showed up once it ran. Roughly in the order they came up.

18 · Engineering Learnings

1 · Config drift bites silently — verify model IDs against the live API

The first migration to W&B Inference used invented model IDs and the wrong base URL, so every call 404'd. The fix wasn't guessing — it was hitting `GET /v1/models` for the exact IDs (`deepseek-ai/DeepSeek-V4-Flash` / `-V4-Pro`). **Lesson:** for any external API, enumerate the real identifiers; don't trust a plausible-looking string (or an LLM's memory of one).

2 · `.env` belongs in `.gitignore` before the first commit

The repo had `.env` committed (and pushed to a public remote) with live keys, even though `.gitignore` listed it — because gitignore doesn't untrack already-tracked files. Rotation, not deletion, is the real fix once a secret is public. **Lesson:** secrets hygiene is a day-zero concern; `git rm --cached` + rotate, and assume anything pushed public is burned.

3 · Env var names are an interface — match the vendor's

Alpaca calls 401'd because the code read `ALPACA_API_KEY` while `.env` used Alpaca's standard `APCA_API_KEY_ID`. Then a second 404 from a doubled `/v2/v2/` because the base URL already included `/v2`. **Lesson:** adopt the vendor's canonical env names, and normalize base URLs defensively.

4 · Guardrails must be enforced in code, not described in a prompt

`min_confidence_to_trade` was "configured," but it lived only in the compliance prompt — an LLM could ignore it. The rebuild moved all hard limits into a pure `evaluate()` function that runs as the final gate and that the LLMs cannot override. It immediately caught a BUY that had been slipping through on a closed market. **Lesson:** anything that must always hold belongs in deterministic code with tests — prompts advise, code enforces.

5 · Fail safe, not open

When the broker is unreachable for guardrail inputs, the trade is blocked, not waved through. **Lesson:** the default on missing safety information is "don't act."

6 · Make LLM output non-fatal

Bare `json.loads(raw)` crashed a whole run on the first formatting drift. The `call_typed` path (extract → Pydantic validate → reprompt once with the error → typed failure) turned a hard crash into a recoverable, observable event. **Lesson:** treat model output as untrusted input — validate, give it one structured chance to fix itself, and degrade typed.

7 · "Advocate" agents don't track the data

The original Bull/Bear were prompted to always argue their side, so the vote was predetermined and only the confidence number moved. Reframing them as data-driven analysts with a lens (look hardest for the upside/downside, but vote honestly) made the votes actually reflect the evidence. **Lesson:** if you want signal, don't hard-code the conclusion into the role.

8 · More agents ≠ more signal; more distinct data does

The instinct to "add voters" is a trap when they all read the same one-line summary — you get correlated echoes and higher cost. The win was giving the existing four distinct inputs (fundamentals, sector relative-strength, macro regime, news). **Lesson:** widen the information, not the headcount.

9 · Duplicated data becomes an anchor

P/E was fed twice — in the price summary and the fundamentals brief — and the agents over-anchored on it as "overvalued." Removing it from the price line plus an explicit "don't over-index on one metric" instruction fixed the bias. **Lesson:** repetition is implicit weighting; say important things once, in context.

10 · Consensus-seeking breeds herding ("3 agree, the 4th folds")

Once three analysts aligned, the holdout capitulated — bandwagon, not analysis. Fixes: tell analysts the headcount is not evidence and a well-reasoned minority beats false consensus; make the Chair protect dissent (push the majority to rebut the minority, never cite the count). **Lesson:** a debate optimized for agreement manufactures agreement — design explicitly for productive disagreement, and expect more tie-breaks.

11 · Give the model a sense of "now"

Agents inferred the date from training data and treated undated prices as live. Injecting a grounded `time_context()` (today's date + market open/closed) and stamping market data with an `as_of` date made horizon reasoning honest. **Lesson:** time-relative reasoning needs an explicit clock; never assume the model knows what day it is.

12 · One source of truth for thresholds

Scattered literals (and a stale `claude-sonnet` config left over from a provider migration) get out of sync. Centralizing every limit + model tier in `config.py` killed the drift. **Lesson:** constants that govern behavior belong in one importable place.

13 · Observability has to be coherent across entry points

Tracing worked everywhere, but metrics only fired from the CLI, a stale duplicate server (`api.py`) shadowed the real one, and the "evaluator" was imported but never called. Deleting the duplicate, logging metrics from the live server too, and replacing the dead scorer made the story consistent. **Lesson:** audit what's actually wired, not what's named — dead/duplicate code is worse than missing code because it misleads.

14 · Inputs are not ground truth — close the outcome loop

The market strip looked authoritative, but it's delayed third-party data with home-rolled indicators, and nothing ever measured whether a decision was right. The decision journal (persist entry price → score against forward price) is the start of real ground truth. **Lesson:** distinguish inputs from outcomes; a system that never grades itself can't be called robust.

15 · Match the indicator to the convention

RSI used a simple moving average and read ~84 where charts showed ~79. Switching to Wilder's smoothing made it comparable to what users actually see. **Lesson:** when you surface a standard metric, compute it the standard way or it quietly misleads.

16 · Robust process ≠ robust system

The harness became hard to crash, well-instrumented, and safety-gated — genuinely robust as software. But with no track record, uncalibrated self-reported confidence, and hallucination risk in the briefs, it is not a robust decision engine. **Lesson:** be precise about which claim you're making; engineering robustness and predictive robustness are different axes.

Recurring themes

- Enforce in code, advise in prompts.
- Validate and ground everything the model says (format, time, and — next — factual claims).
- Design against the failure mode, not just for the happy path (herding, anchoring, stale data).
- Audit the wiring, centralize the knobs, and measure the outcomes.

19 · Limitations & Sponsor Tools

Limitations

- **No proven track record** — the journal can score outcomes, but there's no statistically meaningful sample yet. A robust process is not the same as good predictions.
- **LLM-dependent reasoning** — votes/conviction are self-reported and uncalibrated; agents can still hallucinate figures not present in the research brief.
- **Thin, third-party data** — single source (yfinance), delayed/last-close, home-rolled indicators (RSI uses Wilder's smoothing).
- **No real risk model** — sizing is bounded by caps, not vol-targeting / portfolio risk.
- **Demo-grade ops** — single-user server; no auth; per-run `wandb.init`.
- **Equities only** — no options/derivatives support.

Sponsor tools

Tool	Role in SenseAI
W&B Weave	agent tracing & debate observability
W&B Inference	all LLM reasoning (DeepSeek V4)
Alpaca	paper-trade execution

SenseAI · Sundai Hack #125 · Radhika Khurana & Jeet S Swadia · Licensed under MIT · github.com/khuranaradhika/senseai